

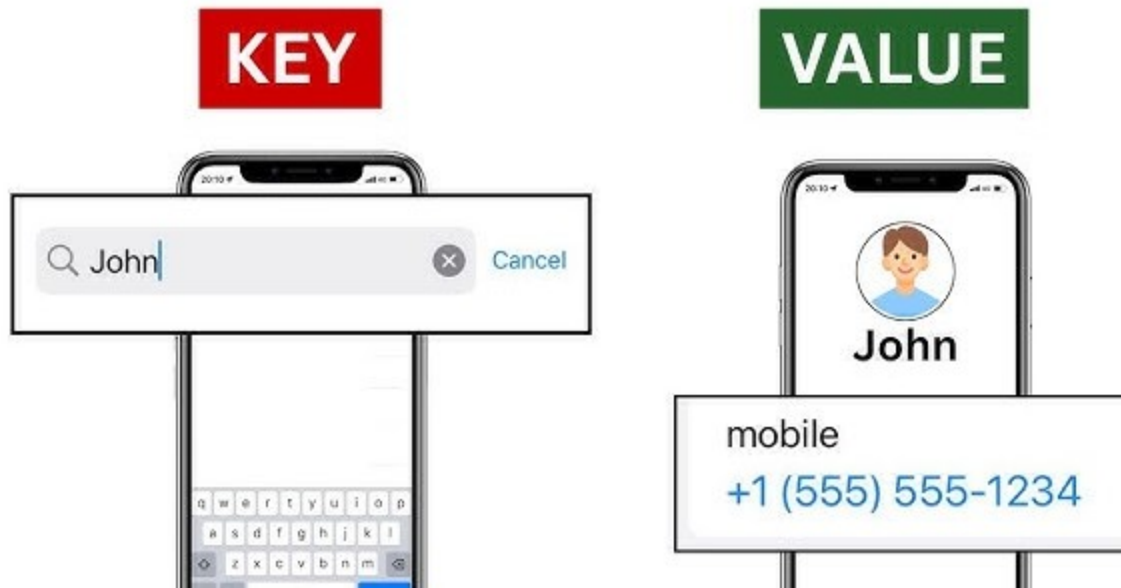
Dictionaries

Eunji Kim

eunjikim@cau.ac.kr

Dictionaries

Dictionaries are used to store data values in key:value pairs.



A dictionary is like a contact list on your phone: each person's name is a **key**, and their phone number is the **value**. You look up the number by using the name, just as you access a value in a dictionary by its key.

Create a Dictionary

```
country_capitals = {  
    'Germany': 'Berlin',  
    'Canada': 'Ottawa',  
    'England': 'London'  
}
```



The diagram illustrates the structure of a dictionary. The opening curly brace '{' is followed by three key-value pairs. Each pair consists of a key (e.g., 'Germany') and a value (e.g., 'Berlin') separated by a colon. Dashed teal arrows point from the text 'element 1', 'element 2', and 'element 3' to the first, second, and third key-value pairs respectively. Below the closing curly brace '}', two purple curly braces are shown. The first brace is under the keys and labeled 'key', and the second brace is under the values and labeled 'value'.

Create a dictionary using curly braces `{}` :

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London"  
}
```

Create an empty dictionary:

```
my_dict = {}
```

or,

```
my_dict = dict()
```

⚠ To create an empty set, you must use the `set()` constructor instead of curly brackets: `my_set = set()`

Access Dictionary Items

Place the **key** inside square brackets `[]` to access the **value** of a dictionary item:

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
    "England": "London"  
}  
  
# access the value of keys  
print(country_capitals["Germany"])    # Output: Berlin  
print(country_capitals["England"])    # Output: London
```

Add Items to a Dictionary

We can add an item to a dictionary by assigning a value to a new key.

For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
  
# add an item with "Italy" as key and "Rome" as its value  
country_capitals["Italy"] = "Rome"  
  
print(country_capitals)
```

Output:

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'Italy': 'Rome'}
```

Remove Dictionary Items

We can use the `del` statement to remove an element from a dictionary.

For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Canada": "Ottawa",  
}  
  
# delete item having "Germany" key  
del country_capitals["Germany"]  
  
print(country_capitals)
```

Output:

```
{'Canada': 'Ottawa'}
```

Change Dictionary Items

Python dictionaries are mutable (changeable). We can change the **value** of a dictionary element by referring to its key.

For example,

```
country_capitals = {  
    "Germany": "Berlin",  
    "Italy": "Naples",  
    "England": "London"  
}  
  
# change the value of "Italy" key to "Rome"  
country_capitals["Italy"] = "Rome"  
  
print(country_capitals)
```

Output:

```
{'Germany': 'Berlin', 'Italy': 'Rome', 'England': 'London'}
```


Iterate Through a Dictionary

We can iterate through dictionary keys one by one using a for loop.

```
country_capitals = {  
    "United States": "Washington D.C.",  
    "Italy": "Rome"  
}
```

Iterate over the keys of the dictionary:

```
for key in country_capitals:  
    print(key)
```

or more explicitly:

```
for key in country_capitals.keys():  
    print(key)
```

Output:

```
United States  
Italy
```

Iterate over the keys of the dictionary to access their corresponding values.

```
for value in country_capitals.values():  
    print(value)
```

Output:

```
Washington D.C.  
Rome
```

Iterate over the (key, value) pairs of the dictionary.

```
for key, value in country_capitals.items():  
    print(f'{key:<15s}: {value}')
```

Output:

```
United States : Washington D.C.  
Italy         : Rome
```

Reminder: List, Tuple, Sets, and Dictionaries

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members (keys).



When choosing a collection type, it is useful to understand the properties of that type.

Summary Table: Elementary Data Types

Python has the following data types by default:

	Data Type	Class	Example	Mutable	Iterable
Atomic	Integer	<code>int</code>	<code>x = 4</code>		
	Float	<code>float</code>	<code>x = 3.142</code>		
	Boolean	<code>bool</code>	<code>x = True</code>		
Sequence	String	<code>str</code>	<code>x = "this" or x = 'this'</code>		O
	List	<code>list</code>	<code>x = [1, 3.3, 'python']</code>	O	O
	Tuple	<code>tuple</code>	<code>x = (6, 'me', False)</code>		O
Collection	Set	<code>set</code>	<code>x = {7, 1, 3, 6, 9}</code>	O	O
	Dictionary	<code>dict</code>	<code>x = {'a': 1, 'b': 2, 'c': 3}</code>	O	O

- A **mutable** object is one whose internal state or value can be changed after it has been created.
- An **iterable** is an object that can be "iterated over," meaning it can return its elements one at a time.

Thank You.